Project Guide

> **NOTE**
>
> This guide is an evolving document. Please refer to the release notes at the end of the page for more information on the updates.

Overview

Your application must have a GUI, a database, API data source(optional), a middle layer connecting your UI to your data. The code should be built within Docker containers so they are portable. Make sure to consider these core requirements when designing the architecture of your application. You are required to use the technology stack that you have learned in this course. If you would like clarifications, please ask your instructor/TA. There is an opportunity to earn extra credit in this group project. Please read that page linked on the sidebar on the left.

Designing your solution

Approaches to designing your solution

Approach	Meaning
Designing at people	The designer defines both the problem and solution without meaningful input.
Designing for people	The designer considers users but retains most decision-making power.
Designing with people	Users participate in defining the problem and shaping the solution.

Approach	Meaning
Designing by people	The community leads; designers contribute tools and technical support.

APPROACH TO PROJECT

Your approach should be **"Designing with people"** for this project.

Steps to designing your solution

To ensure that your application is usable by your intended audience, you should follow the below approach.

Step 1: Pick your application idea

Step 2: Design your solution based on your assumptions

Step 3: Interview your potential users

Step 4: Examine your solution from Step 2 based on the interview in Step 3

Step 5: Redesign with your user's input

Step 6: Present your design to the user and get their feedback

Features in your application

Your application MUST have the following features:

- A login page
- A registration page
- A home page
- A server that allows the UI to communicate with the database
- A database that stores user information
- Passwords must be hashed and stored in the database

- Session Management - The user must be able to log in and out of the application and the session must be maintained
- Application is built within Docker containers - you can find some updates to the docker-compose.yaml in the write-up below.

Some recommended features:

- A UI that allows the user to interact with the application - this can comprise of multiple pages
  - These would be specific to the application you are building
- A profile page (where the user can see their information - optional)
- A superuser/admin account that can manage users (optional)
- A database that stores user data (optional)
  - The purpose would be to store user's activity to be recovered when the user logs back in
- A database that stores data from an API (optional) - if you are requesting repeat data from the API with every call, you could cache some data in your database.

Recommended Directory Structure

For the Project root directory, you should have the following outline:

```
├─<ProjectRepository>/
│   ├─ TeamMeetingLogs
│   ├─ MilestoneSubmissions
│   ├─ ProjectSourceCode
│   ├─ Readme.md
```

Refer to Lab 8 for the contents of the "TeamMeetingLogs" and "MilestoneSubmissions" folder.

In the `ProjectSourceCode` folder in your Github repository for the project, you should have the following structure:

> **.GITIGNORE**
>
> It is essential to include a .gitignore file in your project git repository to avoid tracking build artifacts like node_modules, package-lock.json etc.

```
├─ ProjectSourceCode
|   ├─ docker-compose.yaml
|   ├─ .gitignore
|   ├─ node_modules
|   ├─ package.json
|   ├─ src
|      ├─ views
|         ├─ pages
|            ├─ home.hbs
|            ├─ login.hbs
|            ├─ register.hbs
|         ├─ partials
|            ├─ header.hbs
|            ├─ footer.hbs
|         ├─ layouts
|            ├─ main.hbs
|      ├─ resources
|         ├─ css
|            ├─ style.css
|         ├─ js
|            ├─ script.js
|         ├─ img
|            ├─ home.png
|      ├─ index.js
|      ├─ init_data
|         ├─ create.sql
|         ├─ insert.sql
|   ├─ test
|         ├─ server.spec.js
```

Docker Setup

For the most part, your project will have the same configuration of Docker Containers as the lab assignments. **The docker-compose.yaml file should be in the root directory of your project source code.** The docker-compose.yaml file would look like it is shown below. You can make necessary changes to it.

docker-compose.yaml

```yaml
services:
  db:
    image: postgres:14
    env_file: .env
    expose:
      - '5432'
    volumes:
      - group-project:/var/lib/postgresql/data
      - ./src/init_data:/docker-entrypoint-initdb.d
  web:
    image: node:lts
    user: 'node'
    working_dir: /repository
    env_file: .env
    environment:
      - NODE_ENV=development
    depends_on:
      - db
    ports:
      - '3000:3000'
    volumes:
      - ./:/repository
    command: 'npm start'
volumes:
  group-project:
```

Project source code

You are permitted to reuse any code that was created by any of the team members for the labs in this class. You are also allowed to include code snippets from the provided source code for the labs. All other code should have been originally written by the team members for this project. You may refer to documentation where necessary.

LINKING LOCAL RESOURCES IN YOUR PROJECT

To be able to use local resources that you have created, like css, js, images etc. in your project you will need to add the following line of code, after all the require statements, in the `index.js` file. Please do not include it within a route.

```
app.use(express.static(__dirname + '/'));
```

Once you have added the above line of code, you can use relative paths to link the files.

Project Management

You are expected to continually update your GitHub project board with the progress of your project. You should be adding user stories for each week and their status should be updated as they move along the development lifecycle. Make sure to assign them to team members so it is easier to track individual progress for the members. In Lab 9 we will go over how to use the GitHub project board and for creating and including relevant properties for the user stories. Please refer to that lab as needed to help manage your project board.

Branching on GitHub repository for Development

You are expected to create a new branch for each user story that you are working on. You should NOT be working directly on the main branch. You should be working on a separate branch for each feature. Once you are done with the feature, you should ideally create a pull request to merge the feature branch into the main branch. The pull request should preferably be reviewed by at least one other team member before it is merged into the main branch to ensure that the code is working as expected.

TIP

How do you create a branch for a user story/issue? **Let's check it out**

ReadMe.md

The ReadMe.md should include the following:

1. Brief Application description

2. Contributors - In this case, it will be the team Members
3. Technology Stack used for the project
4. Prerequisites to run the application - Any software that needs to be installed to run the application
5. Instructions on how to run the application locally.
6. How to run the tests
7. Link to the deployed application

You should be updating the ReadMe.md file as you progress through the project. Remember to add/revise sections as you add new features to the application.

Read this **freeCodeCamp article** for guidance on writing a quality ReadMe.md.

This **GitHub repository** has good examples of ReadMe.md.

Automated Test Cases

In Lab 10 you will learn how to test your server endpoints. For the project you will need to test at least 1 additional endpoint, that returns data to the client.

User Acceptance Testing

In Lab 10, you will be writing user acceptance test case scenarios. Once your application is ready and deployed, you are expected to run those scenarios with a suitable audience (could be peers, friends, family) and note the observations and feedback. You will also make a note of what changes, if any, that your team made to incorporate the feedback received from the test audience. These need to be included in your project report.

Deployment Environment

As part of Lab 12, you will be hosting your application. We will teach you how to host your application on **Render** . Please make sure to keep your website active on there for your presentations and until grading is completed for this semester.

GitHub Project Board

- In Lab 9, you will create a GitHub project board and link it to the repository you create for this project. The practices you learnt in Lab 10 will be applied in creating and maintaining the board.
- Make sure that the project board is also **public** since it will be required for grading.
- The board should contain at least 4 columns depicting the stages of development.
- Start by creating a few Epics for the project.
- You will continue adding user stories to the current backlog as you keep advancing through the weeks.

Your board will be monitored by your TA, each week, and you are required to regularly update it as your project progresses.

Weekly Meetings with TA

Each project team is required to meet with their TA at least once a week for ~15 mins. You will be required to submit a weekly update to your TA. All team members MUST be present during the TA meeting.

Purpose

The weekly meetings are meant to be for status update and to make sure you are on track with your project. You should answering the following questions:

- what have you done since the last meeting?
- what are you planning to do until the next meeting?
- what are the challenges you are facing?

If you need help debugging your code or need your TA to take a closer look at your architecture, you can use office hours for that.

Deliverables for the weekly meeting

Before coming to the meeting, you should have the following in your GitHub repository:

- Consistent commits during the week

    - You should be committing your code your feature branch of your repository at least once a day.
    - The commit messages should be descriptive of the changes introduced. **This** resource should be helpful.
    - Your commits should be meaningful - Refer to **this resource** for more information.
- Update your GitHub repository with the latest code.
    - Every team member should have pushed their code to the remote branch of the repository
    - The code should be merged into the main branch of the repository and tested before presenting it to your TA.
    - The features should be fully functional.
        - The scope of the features can be determined by the team.
- Update the status of user stories on the GitHub project board.
- Release notes: A list of updates you have made since the last meeting. This should be in the `Milestone Submissions` folder.
    - Here is are some links that help you write release notes:
        - **Automated Release Notes**
        - **Manually created Release Notes**
- Plan for the following week:
    - What are the features you are planning to implement?
    - To make it manageable, you can create an issue with a checklist on the GitHub project board. **Here** is how you can do that. The checklist should link to the user stories(tracked in GitHub issues) that you intend to work on.

Minutes of the meeting

For each meeting, you will assign a scribe from your team. It is recommended that different members take on this role each week. The scribe will be responsible for taking notes during the meeting and updating the document on the project repository. Points to include in the minutes would be:

1. Decisions made
2. Alternative actions/options discussed
3. Follow-up items, including agreed-on roles and responsibilities You may or may not have content for each of the points above.

The minutes should be updated in the Milestone submissions folder in the project repository at the end of the meeting.

Grading Specifications for weekly deliverables and TA meeting

You will be incrementally graded over the weeks based on the practices used and the progress made on the project. While most of the grading will be for the team effort we are also going to be assessing individual contributions to ensure fairness in grading. Please note that regrades will not be allowed for any of the project related assignments including the weekly evaluations

Each week, the TA will use the following specifications to assign points to the team members. You can see the split between the individual and group evaluations through the rubrics on Canvas.

Criteria	Below Expectations	Meets Expectations
Commits	Student has made less than 3 meaningful commits for the week to the project OR all commits are on the same day	Student has made more than 3 meaningful commits for the week and they are consistently made over a few days
GitHub	Code is not present on GitHub OR code is present on Github on separate branches OR code is in the main branch but it is not functional	All the code completed by the entire team is integrated into the main branch of the repository
Functionality	The code in the main branch is partially broken	The code in the main branch is completely functional
Project Board	GitHub project board doesn't represent all the tasks OR the status of the tasks are not updated	GitHub project board accurately represents the status of tasks the team is working on and plans to work on

Criteria	Below Expectations	Meets Expectations
Release Notes	Release notes are not updated OR not present in the Milestones submissions folder	Release Notes for each week is updated in the Milestones submission folder
Minutes of the meeting (To be submitted on the same day as the meeting)	Minutes of the meeting not included in the Milestones submission folder OR they do not accurately summarize the conversations during the meeting	Minutes of the meeting are uploaded in the Milestones submission folder and they include a summary of the meeting. This may include any/all of the following - the decisions made during the meeting, actions items for team members, and, follow up items
Attendance	Student did not attend the meeting OR student did not contribute anything to the meeting	Student was present at the meeting and actively contributed to the discussion

Recommended Breakdown of the project over the 4 weeks

> **INFO**
>
> This is a recommended breakdown of tasks for the project. You can modify it to fit your team's needs. You should be able to complete the project in 4 weeks. You can use the following checkpoints to make sure you are on track. You can also work vertically across these themes each week - this means you can work on multiple themes in parallel.

User Interfaces

In Lab 8, you will have a chance to work on the wireframes for your application. You should have an outline of the UI pages you intend to use in the application. You may implement your pages

incrementally as part of individual features as you progress through the weeks.

Database and APIs

Database

You should have a database design for your application and have implemented the database. This design will include all the tables, the columns in each, and their data types. Make sure to identify the necessary keys and set the appropriate constraints on the columns in the tables. You will be including this design in the final project report.

External Web APIs

If you intend to use an external API, you should have a plan for how you will use it. You should have a list of the endpoints you intend to use and the data you intend to retrieve from the API. You should also have a plan for how you will store the data in your database, how you will update the data in your database, and how you will handle the rate limits of the API.

> **CAUTION**
>
> If you choose to work with an external API, please make sure to dump the response data from the desired endpoints into either a json file or into your database. This will be crucial in the following scenarios:
>
> - API stops working
> - You are offline
> - API becomes a paid service
> - You've exhausted the rate limit
>
> If you do not know how to do a data dump, please work with a course staff to get that done. This additional step can save you from needing to pivot your project idea or have a non-functional project at the end of the semester.

Application server

At the latest, in this week you should have your server implemented. Your UI should be able to communicate with the Database. You should be able to perform CRUD operations on the database.

Testing

As part of Lab 10, you will create a Test Plan for user testing your application. You will be required to execute that test plan to test your application present your findings in the final project report. The findings should include the following:

Test Cases

A description of the use cases being tested. You should be testing a minimum of 4 use cases. The audience for this activity should be someone outside of your team and preferably the class to receive objective feedback. If you are building an application in a specialized domain, you should be testing it with someone who is familiar with that domain.

Observations

When the users are interacting with your application, you should be observing their actions. You should be taking notes of the following:

- What are the users doing?
- What is the user's reasoning for their actions?
- Is their behavior consistent with the use case?
- If there is a deviation from the expected actions, what is the reason for that?
- Did you use that to make changes to your application? If so, what changes did you make?

Final Presentations

Each team will conduct a live presentation of their project in front of other students, TAs and the instructor in-person. Your team will present the project work during the Week 16 lecture time or lab times. The slots will be made available on Canvas in week 13.

Teams will sign up on Canvas for their chosen presentation date/time. You need to select a time during your lab period. If a time is not available during your lab, you can sign up during a lecture time. Each presentation requires **ALL** team members to be present. If an individual on the team is not present for the presentation, then they will receive a zero for the presentation requirement of the project. If you cannot make it to the presentation, you need to email the TA or instructor before the scheduled presentation.

Content for Presentation

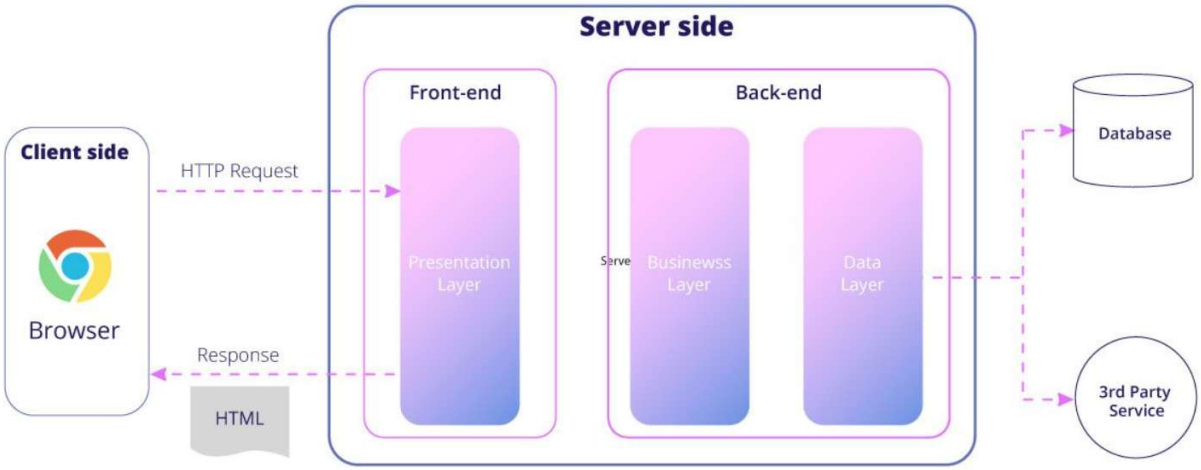
Each team member MUST present some aspect of the project during the presentation. Each team will target 10 minutes for their presentation (no more; no less please.) followed by 5 mins of Q&A. You should present the following items in whatever creative way you wish. At the least, you are expected to have a powerpoint presentation including all the information mentioned below.

- **Title of the project**
- **Names of each person in the group**
- **Description of the project **
    - Think of this as an elevator pitch to sell your product. Alternatively, you could explain your vision.
- **All the tools your group used**
    - Name of the tool, logo, and purpose (e.g., Project Tracking, VCS)
    - Your group's rating on how useful/good this tool/methodology was (ranked 1..5 where 5 stars is best and 1 star is useless)
    - Methodologies (Iterative, agile, pair programming, peer code reviews, other...)
- **Expected tools**
    - Project Tracker (Trello, Asana, or similar)
    - VCS repository (GitHub)
    - Database (PostgreSQL)
    - IDE (e.g. Eclipse, Code::Blocks, xCode.)
    - UI Tools (e.g. Wireframing tools, HTML, HandleBars etc)
    - Application Server (NodeJS)
    - Deployment environment (Render)
    - External APIs (e.g. Google Maps, Twitter etc.)
    - Testing tool (Mocha, Chai, Jest) -
    - Auto-documenter (optional. Doxygen or similar) - *if applicable*
    - Additional tools you may have used - *if applicable*
    - Framework (Express in Node)

- Hardware (e.g. RaspberryPi, Arduino) *if applicable*

- **Architecture Diagram**

  - This diagram should include a detailed diagram depcting the architecture used for your application and the flow of data across the different parts of the architecture. Please find below a sample architecture diagram. Tailor it to specifically suit your project. It must include the tools you used for each layer of the architecture. Make sure to list the names of the external APIs(if any) as well.



- **Challenges**

  - List the challenges you encountered, and how you overcame them and how it may have affected your original project plans.

- **Future Scope/Enhancements**

  - If you were to continue working on this project, what are the key features you would consider integrating to increase its value or applicability in the real world? List at least 3 enhancements.

- **Demo your project**

  - This must be a live working demo. (We highly recommend practicing this demo beforehand.) As a backup, you could record a demo of your project to use during the presentation in case you run into issues with a live demo.

TIPS FOR EFFECTIVE PRESENTATION

1. Because you have a limited amount of time to present, make use of images – remember, a picture is worth a thousand words! Designing good pictures/infographics can really enhance a presentation!
2. You can show a picture of your repository or project tracker, etc. to show how you made use of the tools. Be sure to use a legible font for all text so everyone can read it.
3. Run the presentation by your TA to see if you have everything you need.
4. You can carry brief notes on the topics you want to present. However, plan to use them only as a helper rather than reading from them.

Deliverable

- Create a single PDF for the team
- This submission should be a PDF named ProjectPresentation_[TeamNumber] included in your git repository. This document should be a PDF version of your slides.
- Submit a text file on Canvas with a link to the PDF. Latest due by Dec 4, 11:59 p.m. **No submission on Canvas will result in a "Not Attempted" for the milestone.**
- This milestone will be graded based on the quality of the content during presentation. You are expected to cover all the requirements listed above. It is advisable that each team invests some time in creating the slide deck and (an hour or so) in rehearsing the presentation. You are presenting your effort of 4 weeks here. Choose to tell the story of your project development experience as succinctly as possible. We look forward to seeing your project demos.

Grading Specifications for Project Presentation

Criteria	Below Expectations	Meets Expectations
Presentation	Team did not introduce themselves. Did not clearly describe the application. Presentation was Vague. Unprofessional. Unrehearsed. Went over 10 minutes, or finished too quickly.	Provided Team Name and introduced each person on the team. Everyone spoke. Describe the application, the audience and the problem it solves. Professional

Criteria	Below Expectations	Meets Expectations
		looking. Well rehearsed. Used full 10 minutes.
Challenges	Did not talk about challenges OR mentioned challenges but did not describe solutions	Honestly and openly described the challenges they faced. Described how they overcame the challenges.
Creativity	Interesting application idea. But not quite Fully formed. Started “big” but finished “small”	Pushed technology beyond basic webpage with a database. Used unique and challenging tools. Clever idea.
Technical Details	They mentioned some of the software tools they used in their stack but some were missing. Mentioned agile, but did not talk about the tools they used. They mentioned meetings but did not describe frequency, time, location	They mentioned all the software tools they used in their stack. They mentioned their Project Management method and Project Management task tracking tool. They described effective meeting times & locations.
Product Demo	Showed only one or two features of the app actually working. Partially explained the flow of using the app.	Showed three or more features of the app actually working. Explained the flow of using the app.
Future Scope	The team did not share any enhancements for their application	The team listed at least a few potential enhancements for their application
Q & A	The team was unsure about the answers to some of the questions	The team was able to successfully answer all audience questions, thus demonstrating a clear understanding of the tools used

Peer Evaluation Survey

> **INFO**
>
> **Note: This is a graded survey and accounts for a significant portion of your grade. There is NO replacement assignment or late submissions allowed for this survey.**

- The results of this survey will be used by the instructional staff to calculate grades for the Group project.
- This survey tool allows us to determine the level of participation of each team member. Team members who contributed little to the finished application will see a reduction in their overall score for the project.
- To complete the survey, you must enter your project team number. Your team number consists of your recitation section number plus a digit to identify your team. For example, "101-1". If you do not know your team number, please ask your TA prior to taking the survey.
- You are encouraged to be objective when reviewing your peers' contributions.
- The link to the survey will be provided on Canvas in Week 16 and will be due on Dec 4 at 11:59 p.m. The survey will close after that.
- Each student will use the below criteria for evalluating each member of the team, including themselves, as they fill out the survey:

Criteria	Below	Meets
Meaningful and consistent contributions	Contributions were inconsistent, minimal, or not meaningful.	Made meaningful and consistent contributions to the project.
Communication	Communication was unclear, disrespectful, or not timely.	Communicated clearly, respectfully, and responded in a timely manner.

Criteria	Below	Meets
Reliability and task completion	Did not reliably complete tasks or frequently missed deadlines.	Was reliable and consistently completed assigned tasks on time.
Collaboration and teamwork	Did not collaborate effectively or negatively impacted the team environment.	Collaborated well and contributed positively to the team environment.
Professionalism and openness to feedback	Demonstrated unprofessional behavior or was unreceptive to feedback.	Demonstrated professionalism, respect, and openness to feedback.

- We will look for a majority in the peer evaluations that we receive to assign grade for this assignment, i.e., if the majority of the team members say a student met the above criteria, we will grant a "Meets Expectations" for this student for this assignment.

Project Report

This will be graded based on the amount of effort, the success of the overall project, and use of tools and methodologies throughout project.

You must deploy your project and store your project source code, test cases (UAT), and all relevant project documentation in the project Git repository.

Create a PDF named ProjectReport_[TeamNumber].pdf where team number is the one provided to you on Canvas. Please make sure that your report does not exceed 8-10 pages.

The pdf document must contain the following in the order provided:

Content in report

- **Title:** of the project
- **Who:** Name of each person in the group

- **Project Description:** A 200 word summary of the project
- **Project Tracker - GitHub project board:**
  - Link to your Project Tracker (for instructor & TAs)
  - Screenshot showing your project in your project tracker
- **Video:** 5 minute or less video demonstrating your project. Your audience is a potential customer or person interested in using your product.
- **VCS:** Link to your git Repository. Instructor/TAs will check, weekly, to ensure the following are stored in your VCS repository:
  - Source Code
  - Test Cases
  - Video demo
  - README.md in GitHub
  - Project documentation
  - Project Board
- **Contributions:**
  - A brief (not more than 100 words) from each team member about their contributions.
    - This should include the technologies worked on
    - Features that have contributed to
  - You can also include:
    - A screenshot of the project Board
    - A screenshot of the contributions on GitHub
- **Use Case Diagram:** You need to include a use case diagram for your project. You can build on the use case diagram you created in the proposal. If you built a complete use case diagram for the proposal, you can include it as is.
- **Wireframes** You must include all the wireframes that were created for the project. It is expected that you have at least one for each page. They can be photographs of hand-drawn images.
- **Test results:** In Lab 10, you created a Test Plan. You need to include the test results and observations in the project report. Refer to **this** for more information

- **Deployment:** Link to deployment environment or a written description of how the app was deployed and how one might access/run the app. The app must be live, working, and accessible to your TA.

Be sure to:

- Tag your repo with "Final Submission" (make sure to push your tag to your repo) before your presentation. You can use that for the demo.

> **TIP**
>
> Here is how you can create a release tag

Deliverable

- Create a single PDF for the team
- This submission should be a PDF named ProjectReport_[TeamNumber] included in your github repository by Dec 4 at 11:59 p.m.
- Submit a github link to the report on Canvas.

Grading Specifications for project report

Criteria	Below Expectations	Meets Expectations
Title	Title is missing, unclear, irrelevant, or a placeholder.	Title is present, relevant, and clearly reflects the purpose of the project.
Member Details	Any names or GitHub usernames are missing, mismatched, or incomplete.	All member names and GitHub usernames are included and correctly matched.
Project Description	Exceeds 200 words, unclear, missing major features, or not included.	A concise description (≤ 200 words) summarizing key features and usability.

Criteria	Below Expectations	Meets Expectations
Version Control	Link missing/broken OR repository not organized as required.	Working link to repository is included; repository is organized according to instructions.
GitHub Project Board	Link missing/broken OR user stories absent OR tracking incomplete or incorrect.	Link to project board is provided; user stories created and appropriately tracked
Demo Video	Link missing, broken, or does not demonstrate the project.	Working link to a demo video that effectively showcases the project.
Contributions	Contributions missing, incomplete, or unclear.	Individual contributions for each team member are clearly listed and meaningful.
Wireframes	Wireframes missing, incomplete, or unclear.	Wireframes included for all major pages/screens, showing layout and key elements.
Use Case Diagram	Diagram missing, incomplete, or incorrect.	Complete and logically structured use case diagram illustrating all features.
User Acceptance Test Plan Observations	Observations from real users testing the application are missing, superficial, copied, or misaligned with test scenarios.	Thoughtful observations from real users testing the application are included based on User Acceptance Testing plan from Lab 10.
README.md	Any required section missing, unclear, or incorrect.	README includes: project overview, directory structure explanation, run instructions, and deployed link (if applicable).

Overall grading specifications for the entire project

Here are the 11 items involved in grading the project:

Criteria
Week 1 group contribution
Week 1 individual contribution
Week 2 group contribution
Week 2 individual contribution
Week 3 group contribution
Week 3 individual contribution
Week 4 group contribution
Week 4 individual contribution
Project Presentation
Project Report
Peer Evaluation Points

The grading specifications for each of these are included above and the same can be found on Canvas on each of the assignments. There NO regrades allowed for any of these items.

Release Notes

> **INFO**

> Please refer to this section for a list of any changes that have been made post the release of this guide.